

# Extending SocIoTy to Threshold ECDSA

Angel Coronel  
acoronel@gradcenter.cuny.edu  
City University of New York  
New York, NY, USA

## 1 Abstract

SocIoTy is having an at-home cryptography where users can have their smart home IoT devices help them with security [7]. This is by having cryptographic operations only be possible when the user is in their home. My project is to find out if this work can go from SocIoTy's lightweight primitives to threshold ECDSA signing. We look at two GitHub implementations silence-laboratories/dkls23 and bnb-chain/tss-lib which are based on DKLS23 protocol family and GG18 protocol, respectively. We measure runtime and memory. The main results is that creating a key is what takes the most amount of time and space. This means threshold ECDSA is best used in SocIoTy when key generation is not common and signatures are amortized over a persistent key.

## 2 Introduction

SocIoTy is an idea developed for mitigating a specific risk that comes with storing valuable information on a smartphone. Smartphones, though convenient, are not fully trustworthy since they can be lost, seized, stolen or extracted. Should the phone end up with an attacker, then the attacker can potentially access all of the private data.

SocIoTy tries to force cryptographic operations be tied to the specific context of being at home by having Internet of Things (IoT) devices (smart-home devices) involved in the process. SocIoTy proves that this is practical using a lightweight PRF (pseudo-random function). This project extends this idea of having "at-home cryptography" with a more robust cryptographic primitive, threshold ECDSA signing. ECDSA is already used in real systems like Web3 wallets and digital authentication. So, if ECDSA can be used for SocIoTy, then a user's signing key doesn't need to be in one device. The key can be split across several devices and a signature can be created when enough of those devices cooperate.

In this project, We evaluate two working threshold ECDSA implementations, DKLS23 (implemented in Rust) and Binance tss-lib (implemented in Go). For both implementations, we built local benchmarks for distributed key generation, distributed signing and repeated signing using the same generated key. We measured runtime, memory usage and message bytes when running the protocols. Finally, we analyzed what the results mean on using threshold ECDSA for SocIoTy's system.

The main finding is generating a key takes more resources than signing. This is a reinforced outcome by theory intuition where proving something is much harder than verifying it. To use threshold ECDSA for SocIoTy's system in a real use case, this would mean that key generation is rarely done and the created key shares are used for multiple signing. The results also show the tradeoff in using either implementation. Binance tss-lib had faster runtime measurements but used more memory. DKLS used less memory but

it was slower. This shows that the protocol and the implementation are both important when using this cryptographic primitive.

## 3 Background

### 3.1 Threshold ECDSA

ECDSA stands for Elliptic Curve Digital Signature Algorithm. In normal ECDSA, one private key is made and used by one signer to produce a signature. For an attacker to be successful, they attack the single point of failure. If successful, either by stealing, copying or extracting the key, then the attacker can create signatures as if they were the real user.

Threshold ECDSA is an extension of this by splitting the private key with multiple parties so that no single party has the full private key [4]. Instead, all parties have an equal share of the key and a large enough group of parties can cooperate to produce a valid ECDSA signature.

There are two main phases in threshold ECDSA. The first is distributed key generation (DKG or keygen) which is to create the distributed key shared and produce a public key that can be used by normal ECDSA verifiers. The second phase is distributed signature generation (DSG or signing) which allows a group of parties to cooperate and produce a valid ECDSA signature.

The main parameters are  $n$  and  $t$ .  $n$  is the total number of parties or devices.  $t$  is the threshold parameter. In the signing phase, the system requires  $t + 1$  signing parties for a signature to be accepted. Not all  $n$  parties are needed and  $t$  parties is defined when creating the distributed key. Any combination of the  $n$  parties is valid so long as there are  $t + 1$  parties, order of participants does not matter. In the smart home setting, this matters since not every device needs to be online for every signature.

### 3.2 SocIoTy and "at-home" cryptography

SocIoTy is a system made to have smart home devices support cryptographic operations. This is so the home itself becomes part of the security mechanism to access secure data. In SocIoTy, the phone and home devices work together each with their own layer of cryptographic support. If the phone is taken outside of the home, then the attacker still does not have access to the home devices. If some home devices are compromised, then the attacker does not have access to the phone's secret shares. This makes the system stronger than having the whole key stored in a single place.

IoT devices are useful since they are set to be in the home, common in modern homes and able to distribute trust across multiple devices. A single device is not secure enough to be trusted but a group of devices can be used to store pieces of a secret.

The original SocIoTy design focuses on a lightweight cryptographic primitive, a PRF [7]. PRFs are commonly used to support

applications like two factor authentication and encryption. Threshold ECDSA is a heavier crypto primitive and, theoretically and practically, harder to decrypt.

### 3.3 Protocols studied

In this project, two threshold ECDSA implementations are evaluated. Having both implementations gave way to a useful comparison. They were useful to evaluate if existing threshold ECDSA implementation can support SocIoTy.

The first implementation is DKLS23 from [silence-laboratories/dkls23](https://github.com/silence-laboratories/dkls23) repository on GitHub implemented in Rust [3]. We included it because it has a clean code interface for deploying on systems that made it easy to testing locally and on virtual machines (VMs). DKLS was used for the JetStream cloud part of the project.

The second implementation is Binance tss-lib from [binance-chain/tss-lib](https://github.com/binance-chain/tss-lib) repository on Github implemented in Go. It is based on Gennaro and Goldfeder work on threshold ECDSA [5]. We included it as a well known threshold ECDSA library and as a useful comparison against DKLS.

## 4 Research Question, Scope and Threat Model

### 4.1 Research Question

Seeing how the original SocIoTy paper already covers that lightweight crypto operations can be practical in a smart home setting. This project turns to having that same smart home setting and finding out whether a heavier and more interactive primitive can be used instead. Thus, the main research question is, can real threshold ECDSA implementations support SocIoTy signing when the system is limited by runtime, memory, communication and deployment complexity?

To answer this question, the problem is broken down into smaller chunks. First, we compare the cost of DKG against the cost of DSG. Second, we test whether repeated signing can amortize the setup. Third, we compare DKLS and Binance tss-lib on runtime and memory use. Finally, we find what extra systems work is needed to switch from local simulation to a distributed simulation on the cloud.

### 4.2 Threat Model

As outlined in SocIoTy, the threat model is based on not having a single point of failure by only having a smartphone contain sensitive cryptographic capabilities. In the target setting, the user has a phone and multiple smart home devices. The main security goal is that no single device should hold the full ECDSA private key and, instead, the private key is split across multiple parties. This way, an attacker has to compromise enough devices ( $t + 1$  shares) before they can create signatures.

The main adversary in this project is an attacker who compromises fewer than  $t + 1$  signing parties. This is an attacker that is not able to produce a valid signature alone and the main reason why threshold ECDSA is relevant to SocIoTy.

Additionally, the model considers the risk of the phone being compromised. If the phone is the only device being used for signing, then the phone being compromised will compromise the user's key. In SocIoTy, the phone may coordinate the protocol but it shouldn't have the full private signing key on its own. The home devices

must participate so that a signature can be produced. The home devices being compromised is a separate concern. IoT devices are not strong enough to be considered trustworthy on their own but a single compromised device should not be enough to sign. The system should remain secure as long as the attacker controls fewer than the signing threshold.

A separate concern is availability of the devices themselves. Any device may be offline, busy, replaced or unavailable. The system should not require every device used to create the key to participate in every signature. This works because the system only requires  $t + 1$  parties but if too many devices are unavailable or compromised, then the system may fail to sign.

### 4.3 Scope

The scope of this project is based on what was completed and measured. The completed work includes local benchmarks for DKLS, local benchmarks for Binance tss-lib and distributed cloud orchestration for DKLS on JetStream. The measured data includes runtime, memory use, message bytes, logs and repeated signing behavior. The project also compared implementation tradeoffs between the two protocols tested.

What is not in scope of this project is Binance is not completed as a distributed cloud deployment. Physical Raspberry Pi and ESP32 measurements were not completed. Power consumption was not measured. Device churn, meaning devices added, removed or replaced to the total parties involved, was not tested. This project shows what works, what is expensive and what still needs to be studied.

## 5 System and Experimental Setup

### 5.1 Experiment Goals

The experiments were designed to measure how expensive threshold ECDSA is in practice and different implementations and execution environments reflect that. The biggest comparison is between DKG and DSG. DKG creates the shared signing key and DSG uses the key shares to produce signatures.

The experiments compare local and distributed execution. Local experiments let us see how both implementations compare in similar situations. Distributed DKLS experiments on JetStream let us show what changes when the protocol goes from simulated locally on one machine (with one processor) to separate machines.

### 5.2 Local Benchmark Environment

Both DKLS and Binance tss lib were run locally. In the local setting, parties were simulated on the same machine. This made it possible to test several configurations quickly and compare the two implementations under controlled conditions.

The local experiments included distributed key generation, distributed signing, and repeated signing. Distributed key generation was run to create the shared key material. Distributed signing was then run to produce signatures. In the repeated signing experiments, the same generated key was reused for multiple signatures. This design was important because it tested whether the expensive setup phase could be spread across several signing operations.

The main configurations tested were:

- (3, 2)
- (5, 2)
- (5, 3)
- (10, 5)
- (10, 6)
- (10, 7)
- (10, 8)

For repeated signing, we tested (3, 2) and (5, 3) with 1, 5, and 10 signatures. We also tested fixed and random signing modes. In normal mode, it is the same parties doing the repeated signing rounds. In random mode, it is a random combination of parties for each round of signing. In each signing experiment, the number of signers was based on the threshold rule, so signing used  $t$  plus 1 parties instead of all  $n$  parties. This matches the important threshold ECDSA property that a quorum can sign without requiring every device to be online.

### 5.3 DKLS Cloud and JetStream Environment

After both protocols were proven to work and were benchmarked locally, we build a distributed DKLS setup on JetStream. The goal was to go a step further from local simulation and test DKLS in an environment each party runs on a separate VM.

We had a total of 11 VMs on jetstream. There was one VM set up to be a controller for orchestration and the rest as party VMs. The controller coordinated the experiment and was the only one with 2 cores of GPU, 6 GB of RAM and a total of 20 GB of disk. The 10 remaining party VMs participated in the DKLS protocol each with only 1 CPU core, 3 GB of ram and total of 20 GB of disk. To support communication between the machines, we build a TCP network layer and relay setup for DKLS runs.

The DKLS cloud work included running DKG, saving the generated key shares, running DSG and collecting metrics from each run. The party VMs were purposefully given constrained resources to simulate real world deployment as close as possible. This does not prove that DKLS will work on real IoT devices but its a meaningful step towards having SocIoTy be deployed on a real environment.

### 5.4 Protocol Execution Model

Each experiment has a division between the cost of setup and signing cost. The setup phase is DKG done first to create the key shares. DSG is done after to produce signatures with the signing parties. In the repeated signing experiments, the same key is reused on multiple signatures.

## 6 Methodology

### 6.1 DKLS Methodology

For DKLS, we followed and used the examples and metrics provided in the repository to measure the cost of DKG and DSG [2]. We treated DKG and DSG as separate parts of the system to make it possible in finding which part of the protocol was the main bottleneck.

In the local DKLS experiments, we collected runtime, memory usage and message bytes, The message and byte measurements

show the communication cost while the memory and runtime measurements show whether the protocol could eventually be used on a resource constrained IoT device.

After testing locally, we extended the DKLS work to a distributed JetStream setup. This required moving from a local simulation into a cloud environment where each party runs on their own dedicated VM. I had the controller VM coordinate the experiments with dedicated scripts for each experiment so that the parties could exchange protocol messages across the network. The controller VM has access to the party VMs by manually configuring their internal IP addresses within the JetStream network. We also had a TCP communication layer so that the parties could exchange protocol messages across the network. Part of the experiments involved having each party report back to the controller VM what each experiment cost was in terms of runtime and memory for the specific party. The controller VM was in charge of storing each parties results for later analysis. This allowed us to see DKLS in a more realistic setting with multiple machines having to collaborate.

### 6.2 Binance Methodology

For Binance tss-lib, we mimicked the methodology used for DKLS with local benchmarks to test DKG, DSG and repeated signing. The goal was to compare them side by side since Binance's implementation is a popular threshold ECDSA implementation. The Binance experiments measured runtime and memory usage. Like DKLS, DKG and DSG were separated to compare the cost of each. We also tested repeated signing.

In this project, Binance was only completed as a local benchmark. We did not complete a distributed cloud deployment for Binance. From the code available in their public repository, the library leaves more of the network and application structure of the user of the library. Meaning, more application level networking would be needed to run the protocol fairly across multiple VMs. This detail matters since Binance cannot be treated with the same level of results as the distributed DKLS cloud run.

### 6.3 Metrics

The experiments collected several types of measurements. Each metric helped to navigate and answer different parts of the research question.

RSS is not the only memory used by the crypto protocol itself since it also includes implementation and runtime overhead. This especially impacts Binance results because RSS can include Go runtime overhead besides the protocol state. Thus, the results should be used for implementation measurements not pure mathematical protocol memory.

## 7 Results

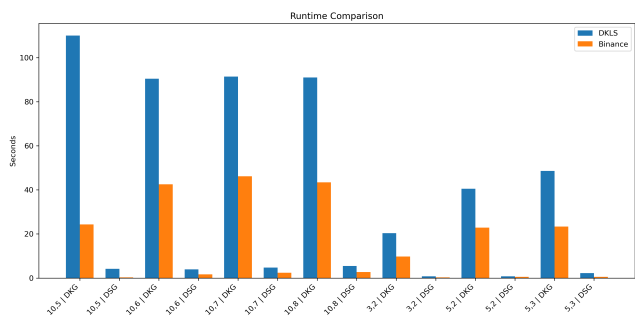
### 7.1 Local Runtime: DKG Dominates Signing

The most straightforward result from the experiments is seeing DKG cost much more than DSG. This pattern repeated itself all tests and became a main finding for the project. DKG is the setup followed by signing so in practice, DKG should be avoided as much as possible.

In the local run, DKLs had a higher runtime than Binance. Binance was faster overall in runtime but both had the same general

**Table 1: Metrics collected during threshold ECDSA evaluation**

| Metric                   | Meaning                                                | Why it matters                                                   |
|--------------------------|--------------------------------------------------------|------------------------------------------------------------------|
| Runtime                  | Time required for DKG, DSG, or repeated signatures     | Shows latency and identifies bottlenecks                         |
| RSS / memory             | Maximum resident memory usage during the run           | Shows whether weaker devices may be able to support the protocol |
| Message bytes            | Amount of communication sent and received              | Shows network overhead                                           |
| Logs and artifacts       | Party outputs, run records, and saved experiment files | Supports debugging and reproducibility                           |
| Repeated signing average | Average signing cost after one key generation          | Shows whether setup cost can be amortized                        |



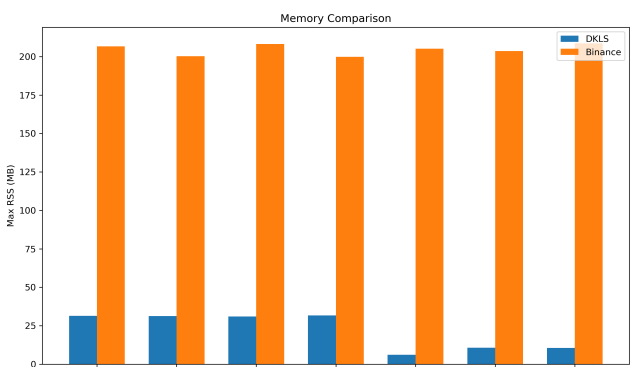
**Figure 1: Runtime comparison between DKLS and Binance threshold ECDSA across DKG and DSG phases. The chart separates setup and signing costs, showing that DKG dominates the total runtime while DSG signing is comparatively smaller.**

pattern of key generation taking taking much longer than signing. On SocIoTy, this means that a use would have to wait much longer if they wanted to create a key whenever they want to sign a transaction or authentication.

## 7.2 Memory Usage: Binance Uses More Memory

The memory results show a different tradeoff from the runtime results. From the local runs, we can see that Binance was faster but used more memory than DKLS. For constrained devices, this would make a huge difference since total memory usage might become a bigger problem than raw speed.

Overall, DKLS had better memory efficient measurements. Binance might have a higher memory usage from a combination of the protocol implementation plus Go runtime overhead. The trade-off is important relative to the devices that would have to run these protocols. If local speed is important, then Binance would be faster than DKLS. If memory is important, than DKLS would take up less space than Binance. This becomes relevant when deploying either protocol on microcontrollers like Raspberry Pi Zero or ESP32.



**Figure 2: Memory comparison between the DKLS and Binance threshold ECDSA implementations. Across the tested configurations, DKLS uses much less maximum resident memory than the Binance implementation.**

## 7.3 Repeated Signing Results: Reusing Keygen Works Well

The repeated signing experiments tested reusing the same key generated multiple times in signing to see how runtime and memory costs change. To mimic a real world environment, a fix mode and random mode with the same signer subset in each round or a random signer subset in each round. The repeated signing was done with 1, 5 and 10 signatures after a single key is generated.

Repeated signing showed that signing remains relatively stable using the same key regardless of a fixed or random subset signers. Binance continued to have the best overall runtime.

Repeated signing is the most relevant result to SocIoTy for deployment of threshold ECDSA. At home signing would be more useful when a user only has to make a new key occasionally. Frequenting the same key with multiple uses with periodical key flushes or as needed would have the best overall use of resources and security.

## 7.4 DKLS on JetStream: Distributed Cloud Execution

Running DKLS on cloud VMs is the main distributed systems part completed for this project. The JetStream results showed DKLS

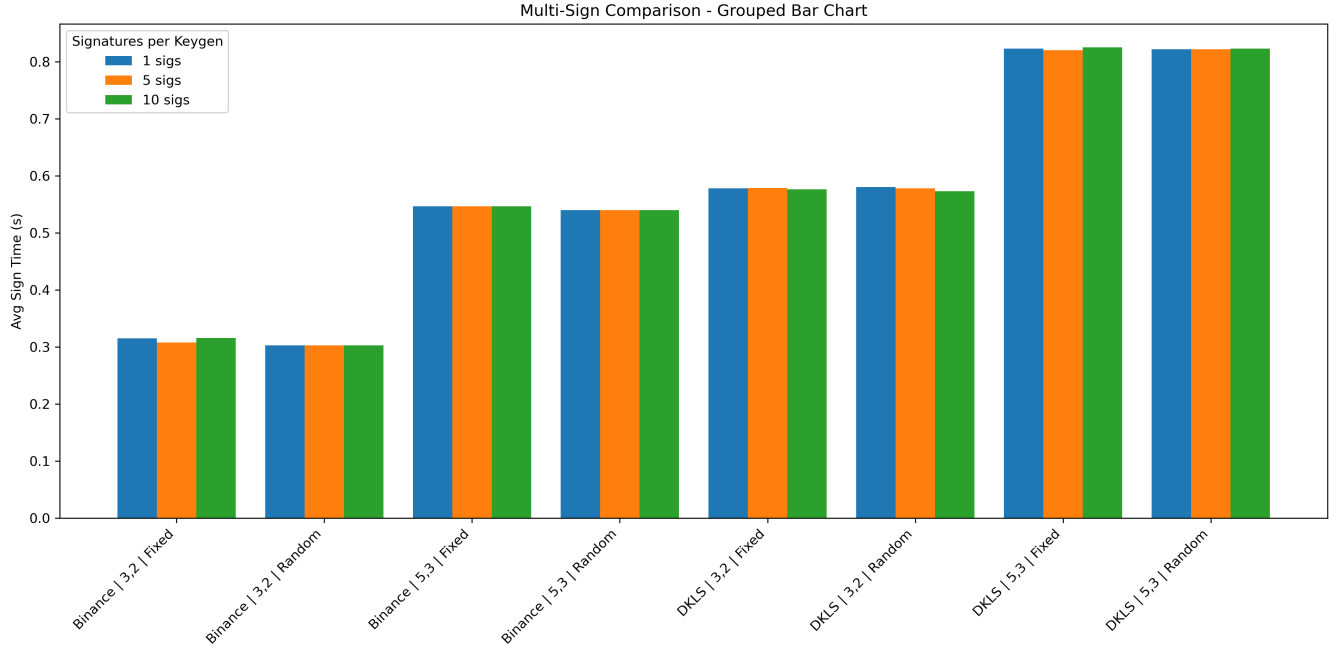


Figure 3: Average signing time for repeated signing experiments after one key generation step. The grouped bars compare fixed and random message signing across different party and threshold configurations, showing how signing cost behaves when multiple signatures are produced from the same setup.

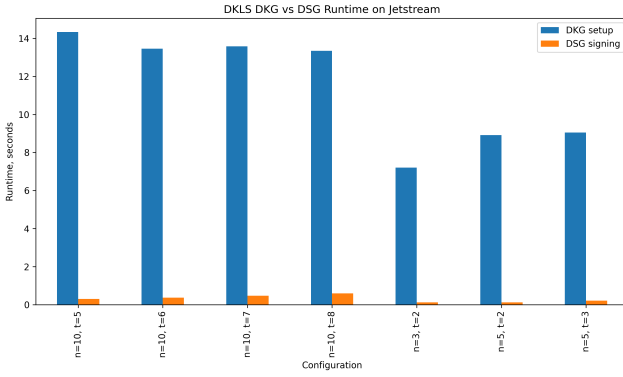


Figure 4: DKLS DKG setup time compared with DSG signing time on the Jetstream virtual machine environment. The figure shows that DKG is the more expensive setup phase, while DSG signing is much faster once the key generation step is complete.

could complete the cloud workflow for the configurations tested locally. Staying consistent with local tests, DKG remained more expensive than DSG. Having DKLS run a multiple machines added networking overhead but has less time overall. This may be due to local testing running on one core processor where, in the case of JetStream, each VM has their own dedicated processor where each runs the protocol concurrently.

## 7.5 DKLS vs Binance: Systems Tradeoff

In comparing both protocols, we find that there is not a clear choice but only a matter of how each one best fits a use case. Binance performs better in runtime while DKLS has better memory usage and proved to be capable of being moved into a distributed cloud experiment. This means choosing one depends on the practicality of the system they will be used for.

DKLS was a good fit for the actual implementation of JetStream. DKLS helped strengthen the point that threshold ECDSA is a feasible protocol for SocIoTy by being moved from local execution to JetStream. This does not mean that Binance tss-lib is not, there is more system work required so that it can be tested fairly. Binance library has clear crypto protocol logic but a distribution system still needs networking, coordination and message handling for further testing.

This highlights the point that local testing is not enough to judge feasibility of threshold ECDSA on SocIoTy. Deployment complexity should also be considered as well as different protocols which provide the same cryptographic capabilities.

## 8 Discussion

### 8.1 What the Results Mean for SocIoTy

The results thus far show success on extending threshold ECDSA as a cryptographic primitive for SocIoTy as long as the DKG is an infrequent operation and signing is the normal recurring operation. From the experiments, DKG is the bottleneck for both protocols.

This matters since a user should not have to run the setup phase every time they want to sign something. If every signature requires a new key then the system would be too slow and expensive for regular use. The user would only experience the lower signing cost. A user can run DKG when setting up the smart home system then the devices would keep their key share and participate in signing when needed. The expensive work would only have to be repeated when the device group changes or when the key is old and needs regenerating.

This is also why device churn matters. Realistically, smart home devices can be added, removed, replaced, turned off or disconnected at any given moment and for any number of reasons. Future versions would need to make sure to account for device churn to keep the overall system efficient.

The results show the tradeoff between both protocols. In the setting of SocIoTy, the goal is to understand which is more appropriate. A fast protocol but consumes a lot of memory is convenient but may not be appropriate for weaker devices. Microcontrollers do not have the same memory capacity as a laptop or cloud VM. Because of that, a slower but more memory efficient implementation could be more suitable for the leap of using smart home devices to run threshold ECDSA.

## 8.2 Local Simulation vs Distributed Reality

The local benchmarks are useful since they make it easier to compare both protocol implementations in a controlled environment. This made it simpler to have nearly identical configurations on tests and measurements without adding network complexity. This is why local benchmarks were the first step.

However, in a real system, the distributed deployment of the protocols is more reliable for testing than simulating the parties inside one machine. There would be several devices within a network like all connected to the WiFi. This adds latency, message routing, coordination and failures that local testing does not include.

Furthermore, although DKLS JetStream deployment is a step closer to the real setting, it still does not replace physical IoT testing. Local tests are a proof of concept and show the basic cost of implementation of the protocols. The cloud tests show how the protocols are orchestrated on separate machines. Finally, physical IoT tests would be the next step to determine if the same can be done on hardware.

## 9 Limitations

This project has some clear limitations. The most important one is the lack of physical device testing. The hardware was purchased but were not setup and tested as part of the completed results. The second most important is the lack of a deployed implementation of Binance using multiple VMs on JetStream. All claims on results with Binance are reduced to local testing which weakens its claim of how suitable it is for SocIoTy.

Other limitations are already discussed throughout the paper. Namely, cloud VMs are not the same as physical IoT devices. VMs do not reproduce the same memory, CPU, storage or power constraints as Raspberry Pis or ESP32 microcontrollers. Another is power consumption is not measured with any of the final results

gathered. This is important because in a real environment, a given device may also be constrained in energy consumption.

## 10 Future Work

### 10.1 Finish Binance Cloud Deployment

The immediate step that follows is using the already created cluster of VMs on JetStream to complete the Binance cloud deployment tests. A networking layer or wrapper would be needed to orchestrate this. An orchestration script would be needed to run and measure similar metrics to what is already done for DKLS. This would make the comparison of both protocols stronger because they would be tested in distributed settings.

### 10.2 Run on Real IoT Hardware

After both protocols have been implemented in a distributed, controlled setting using JetStream, they can both be run on real hardware. The devices used for testing involve direct comparisons to real devices used in homes like Raspberry Pi, Raspberry Pi Zero, and ESP32 devices as shown in [7]. Real devices have weaker CPUs, less memory, smaller storage and different network behavior that would need further work to strengthen the claims made in this paper.

### 10.3 Measure Power Consumption

Future experiments involving real hardware should measure the power draw in all tests. Real IoT devices have a smaller energy usage than most home devices but it is non negligible. There's a chance that a protocol might finish to completion within a single device but still to expensive based on energy consumption or may interrupt a normal function of the device. This would help compare energy consumption of both implementations.

### 10.4 Map Bottlenecks Back to Protocol Internals

A different angle to this project would be to connect what the measured bottlenecks for runtime and memory are and connect them back to the cryptographic subprotocols as defined in the published papers. For DKLS, this might involve a deeper look into OT and RVOLE based multiplication [1, 3]. For Binance, this might involve a deeper look into Paillier based operations, preparameter generation and related proof steps [6].

Connecting the systems measurements back to the theory might also involve looking at the implementation of the protocols deeper. Identifying which protocol components, in code and theory, can be pointed at bottlenecks. This can be further studied by cryptographers in optimizing the underlying crypto protocols.

## 11 Conclusion

In conclusion, this project extended the research done in SocIoTy by evaluating if threshold ECDSA can fit the "at-home" cryptography model. The original work shows that PRFs, a lightweight cryptographic primitive, can be practical with smart home devices. This project studied whether a heavier and more interactive cryptographic primitive can support the same goal.

The experiments show the distributed key generation is the main bottleneck while signing, including repeated signing, is cheaper

and more stable. For the context of SocIoTy, this means threshold ECDSA is best suitable when setup is less frequent than signing.

Two protocols, DKLS and Binance tss-lib, were used throughout the testing of this project. Local results show there's a direct tradeoff of runtime and momery usage of both. Binance was faster but used more memory whereas DKLS was slower but used less memeory. DKLS was implemented on a distributed cloud environment using multiple VMs which showed how distrubution complexity matters in evaluating threshold ECDSA for SocIoTy.

Overall, the results suggest that threshold ECDSA is posisble for SocIoTy but with the correct considerations. The system should avoid generating keys often and reuse key shares as much as possible.

## References

- [1] Carsten Baum, Samuel Dittmer, Peter Scholl, and Xiao Wang. 2023. SoK: Vector OLE-Based Zero-Knowledge Protocols. Manuscript dated June 7, 2023.
- [2] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. [n. d.]. Threshold ECDSA in Three Rounds. Presentation slides. Slide deck, available from dkls.info.
- [3] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. 2024. Threshold ECDSA in Three Rounds. In *Proceedings of the 45th IEEE Symposium on Security and Privacy (S&P)*. <https://eprint.iacr.org/2023/765> Cryptology ePrint Archive, Paper 2023/765.
- [4] Jack Doerner, Yashvanth Kondi, Eysa Lee, abhi shelat, and LaKyah Tyner. 2023. Threshold BBS+ Signatures for Distributed Anonymous Credential Issuance. In *Proceedings of the 44th IEEE Symposium on Security and Privacy (S&P)*.
- [5] Rosario Gennaro and Steven Goldfeder. 2018. Fast Multiparty Threshold ECDSA with Fast Trustless Setup. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 1179–1194.
- [6] Niv Gilboa. 1999. Two Party RSA Key Generation. In *Advances in Cryptology – CRYPTO '99 (Lecture Notes in Computer Science, Vol. 1666)*. Springer, 116–129.
- [7] Tushar M. Jois, Gabrielle Beck, Sofia Belikovetsky, Joseph Carrigan, Alishah Chator, Logan Kostick, Maximilian Zinkus, Gabriel Kapchuk, and Aviel D. Rubin. 2024. SocIoTy: Practical Cryptography in Smart Home Contexts. *Proceedings on Privacy Enhancing Technologies* 2024, 1 (2024), 447–464. doi:10.56553/popets-2024-0026